

GPIO 輸入/輸出與狀態機實作

學號/姓名

- B11413130 葉峻岑
- B11413126 王盛弘

- 開發環境 STM32CubeIDE / HAL Library

01

LAB

執行結果影片連結

- <https://youtu.be/m2znB5O2zBg?si=qjylGldrkJLYqcZK>

狀態圖

主程式流程

讀取 Button

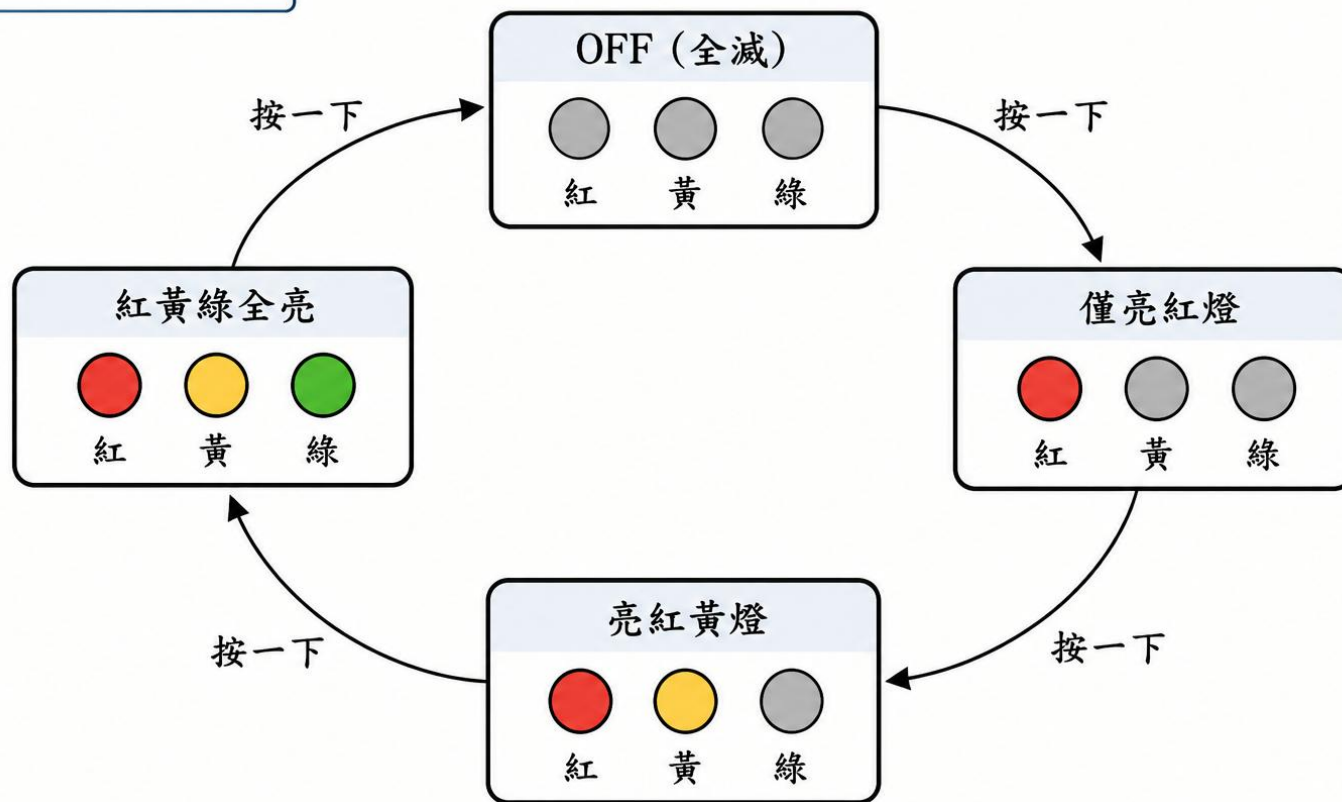
偵測一次按下

切換狀態

更新 LED



LED 狀態機 - 狀態轉移圖



生成 Code 的 Prompt

第一部份

我的現況 (Context): 我目前有 NUCLEO-G474RE 開發板、麵包板、3顆 LED（紅黃綠）和限流電阻，開發軟體使用 STM32CubeMX 與 STM32CubeIDE。

我的需求 (Task): 我需要實作一個功能：每次按下板子上的 User Button，LED 燈號需依序發生變化並循環（1.僅亮紅 -> 2.亮紅黃 -> 3.亮紅黃綠 -> 4.全滅 -> 5.回到步驟1）。請幫我列出適合的 GPIO 腳位規劃與基礎硬體接線邏輯。

邊界條件 (Constraints): 本次作業嚴禁使用 PC5、PC6、PC8 腳位。

第二部份

我的現況 (Context): 我已經決定好硬體配置：按鈕使用內建的 PC13，紅、黃、綠 LED 分別外接在 PA9、PC7、PB6 腳位。

我的需求 (Task): 請指導我如何在 STM32CubeMX 軟體中，針對這四個腳位進行正確的 GPIO 狀態設定，例如 Input/Output，以及如何設定 User Label 以利後續開發。

邊界條件 (Constraints): 請將說明CubeMX 的介面操作與設定步驟。

第三部份

我的現況 (Context): 我已經在 CubeMX 完成腳位設定，並且生成了專案程式碼，目前進入 STM32CubeIDE 的 main.c 檔案準備撰寫邏輯。

我的需求 (Task): 為了達成按鈕切換燈號的功能，我需要實作按鈕防彈跳以及狀態機。請以步驟說明形式，教我這個主迴圈的邏輯判斷順序，以及我需要宣告哪些狀態變數來記錄按鈕次數。

邊界條件 (Constraints): 為了讓我能自己練習語法，請使用流程說明、條件式列表或虛擬碼（Pseudocode）來向我解釋邏輯架構。

程式碼

```
/* USER CODE BEGIN PV */  
  
uint8_t step = 0;    // 記錄目前 LED 的狀態切換步驟 (0~4)  
  
uint8_t btn_last = 0; // 記錄上一次的按鈕狀態，用來比對是否剛被按下  
  
/* USER CODE END PV */  
  
/* USER CODE BEGIN WHILE */  
while (1)  
{  
    // 1. 讀取現在的按鈕電位 (開發板 PC13 按下時為 High)  
    uint8_t btn_now = (HAL_GPIO_ReadPin(BTN_GPIO_Port, BTN_Pin) == GPIO_PIN_SET);  
  
    // 2. 偵測按鈕是否剛被按下 (上升緣觸發：上次為 0，這次為 1)  
    if (btn_now == 1 && btn_last == 0)  
    {  
        // 3. 軟體防彈跳：延遲 20 毫秒，避開機械按鍵接觸瞬間的雜訊  
        HAL_Delay(20);  
  
        // 4. 再次確認按鈕確實被按著，排除誤觸可能  
        if (HAL_GPIO_ReadPin(BTN_GPIO_Port, BTN_Pin) == GPIO_PIN_SET)  
        {  
            step++; // 確定按下，進入下一個步驟
```

程式碼

```
// 5. 狀態循環限制：第5次按下時，重置回步驟 1
```

```
if (step > 4)
```

```
{
```

```
    step = 1;
```

```
}
```

```
// 6. 依照目前的 step 狀態，控制對應的 LED 亮滅
```

```
switch (step)
```

```
{
```

```
    case 1: // 狀態 1：只點亮紅色 LED
```

```
        HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin, GPIO_PIN_SET);
```

```
        HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin, GPIO_PIN_RESET);
```

```
        HAL_GPIO_WritePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin, GPIO_PIN_RESET);
```

```
        break;
```

```
    case 2: // 狀態 2：點亮紅色、黃色 LED
```

```
        HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin, GPIO_PIN_SET);
```

```
        HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin, GPIO_PIN_SET);
```

```
        HAL_GPIO_WritePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin, GPIO_PIN_RESET);
```

```
        break;
```

程式碼

```
case 3: // 狀態 3：紅、黃、綠三顆 LED 全亮
```

```
    HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin, GPIO_PIN_SET);  
    HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin, GPIO_PIN_SET);  
    HAL_GPIO_WritePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin, GPIO_PIN_SET);  
    break;
```

```
case 4: // 狀態 4：三顆 LED 恢復初始狀態全滅
```

```
    HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin, GPIO_PIN_RESET);  
    HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin, GPIO_PIN_RESET);  
    HAL_GPIO_WritePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin, GPIO_PIN_RESET);  
    break;
```

```
}
```

```
}
```

```
}
```

```
// 7. 將這次讀取到的按鈕狀態存起來，給下一次迴圈比對使用
```

```
btn_last = btn_now;
```

```
/* USER CODE END WHILE */
```

```
/* USER CODE BEGIN 3 */
```

```
}
```

```
/* USER CODE END 3 */
```